Phala Network TEE 구현체 분석 - Pherry (릴레이어 시스템)

1. Pherry 개요

Pherry는 **블록체인과 TEE 환경을 연결하는 릴레이어**로, 블록체인의 상태를 TEE로 전달하고 TEE의 실행 결과를 블록체인에 반영하는 핵심 브리지 역할을 수행합니다.

2. 핵심 구성 요소

2.1 의존성 구조

```
# 비동기 런타임
├── tokio: 비동기 런타임 (full features)
├── futures: Future 추상화
├── async-stream: 비동기 스트림

# 네트워킹
├── reqwest: HTTP 클라이언트 (rustls-tls, socks)
├── serde_json: JSON 직렬화

# Substrate 통합
├── sp-core: Substrate 핵심 프리미티브
```

```
├── sp-runtime: 런타임 타입
├── sp-trie: 머클 트라이
├── sp-consensus-grandpa: GRANDPA 합의
├── sc-consensus-grandpa: GRANDPA 클라이언트
├── codec: SCALE 코덱
├── hash-db: 해시 데이터베이스

# Phala 특화
├── phala-types: Phala 타입 정의
├── phala-pallets: Phala 팰릿
├── phala-trie-storage: 트라이 스토리지
├── phala-node-rpc-ext: RPC 확장
├── phactory-api: Phactory API 클라이언트
├── phaxt: Phala 트랜잭션 라이브러리
└── sgx-attestation: SGX 증명 (report 기능)
```

3. 주요 기능

3.1 블록체인 모니터링

```rust
// 블록 모니터링 예시 (추정)
async fn monitor_blocks() -> Result<()> {
let client = SubstrateClient::new().await?;
```

```
let mut block_stream = client.subscribe_blocks().await?;
while let Some(block) = block_stream.next().await {
let block_hash = block.hash();
let block_number = block.number();
// TEE로 블록 정보 전달
relay_block_to_tee(block_hash, block_number).await?;
// 상태 변경 사항 처리
process_state_changes(&block).await?;
}
Ok(())
}
```

3.2 상태 동기화

- **헤더 동기화**: 블록 헤더 정보 전달
- **상태 루트**: 상태 트라이 루트 업데이트
- **저스티피케이션**: GRANDPA 합의 증명 전달
- **이벤트 필터링**: 관련 이벤트만 선별 전달

4. TEE 통신 인터페이스

4.1 Phactory API 클라이언트

```
// Phactory 통신 예시 (추정)
struct PherryClient {
```

```
client: reqwest::Client,

base_url: String,

}



impl PherryClient {

async fn sync_header(&self, header: Header) -> Result<SyncResponse> {

let response = self.client

.post(&format!("{}/sync_header", self.base_url))

.json(&SyncHeaderRequest { header })

.send()

.await?;

response.json().await

}

async fn dispatch_block(&self, block: Block) -> Result<DispatchResponse> {

let response = self.client

.post(&format!("{}/dispatch_block", self.base_url))

.json(&DispatchBlockRequest { block })

.send()

.await?;

response.json().await

}

}
```

4.2 메시지 타입

- **SyncHeader**: 블록 헤더 동기화
- **DispatchBlock**: 블록 처리 요청
- **SyncParaHeader**: 파라체인 헤더 동기화
- **SyncCombinedHeaders**: 통합 헤더 동기화

5. 트랜잭션 처리

5.1 phaxt 라이브러리 활용

```
// 트랜잭션 제출 예시 (추정)
async fn submit_transaction(tx_data: Vec<u8>) -> Result<TxHash> {
let client = PhaxtClient::new().await?;
let tx = client.create_transaction(tx_data)?;
let tx_hash = client.submit_transaction(tx).await?;
// 트랜잭션 확인 대기
client.wait_for_finalization(tx_hash).await?;
Ok(tx_hash)
}
```

5.2 트랜잭션 유형

- **워커 등록**: 새로운 TEE 워커 등록
- **하트비트**: 워커 생존 신호

- **상태 업데이트**: TEE 실행 결과 반영
- **증명 제출**: Attestation 결과 제출

6. 합의 메커니즘 통합

6.1 GRANDPA 지원

```
// GRANDPA 저스티피케이션 처리 (추정)
async fn process_justification(justification: GrandpaJustification) -> Result<
// 저스티피케이션 검증
verify_justification(&justification)?;
// TEE로 전달
let tee_client = PhactoryClient::new();
tee_client.sync_justification(justification).await?;
Ok(())
}
```

6.2 합의 상태 추적

- **Finalized Blocks**: 최종 확정된 블록 추적
- **Best Blocks**: 최선 블록 체인 추적
- **Fork Detection**: 포크 감지 및 처리
- **Reorg Handling**: 체인 재구성 처리

7. 에러 처리 및 복구

7.1 연결 관리

```
// 연결 복구 로직 (추정)
async fn maintain_connection() -> Result<()> {
let mut retry_count = 0;
const MAX_RETRIES: u32 = 5;
loop {
match connect_to_tee().await {
Ok(connection) => {
retry_count = 0;
handle_connection(connection).await?;
}
Err(e) if retry_count < MAX_RETRIES => {
retry_count += 1;
let delay = Duration::from_secs(2_u64.pow(retry_count));
tokio::time::sleep(delay).await;
}
Err(e) => return Err(e),
}
}
}
```

7.2 상태 일관성 보장

- **체크포인트**: 주기적 상태 체크포인트
- **롤백**: 오류 시 이전 상태로 복구
- **재동기화**: 전체 상태 재동기화
- **무결성 검증**: 상태 무결성 주기적 검증

8. 성능 최적화

8.1 배치 처리

```
// 배치 처리 예시 (추정)
async fn batch_sync_headers(headers: Vec<Header>) -> Result<()> {
const BATCH_SIZE: usize = 100;
for chunk in headers.chunks(BATCH_SIZE) {
let batch_request = BatchSyncRequest {
headers: chunk.to_vec(),
};
tee_client.batch_sync_headers(batch_request).await?;
}
Ok(())
}
```

8.2 캐싱 전략

- **헤더 캐싱**: 최근 블록 헤더 메모리 캐싱
- **상태 캐싱**: 자주 접근하는 상태 캐싱

- **결과 캐싱**: 처리 결과 임시 저장

9. 모니터링 및 로깅

9.1 메트릭 수집

- **처리량**: 초당 처리 블록 수
- **지연시간**: 블록 처리 지연시간
- **에러율**: 오류 발생 비율
- **연결 상태**: TEE 연결 상태

9.2 로깅 시스템

```
// 로깅 예시 (추정)
use log::{info, warn, error};

async fn process_block(block: Block) -> Result<()> {
info!("Processing block #{}", block.number());
match sync_block_to_tee(&block).await {
Ok(_) => info!("Block #{} synced successfully", block.number()),
Err(e) => {
error!("Failed to sync block #{}: {}", block.number(), e);
return Err(e);
}
}
```

```
Ok(())

}
```

10. 설정 및 배포

10.1 명령행 인터페이스

```
# Pherry 실행 예시

pherry  --substrate-ws-endpoint  ws://localhost:9944 \

--pruntime-endpoint http://localhost:8000 \

--mnemonic  "your mnemonic phrase here" \

--fetch-blocks 100
```

10.2 환경 설정

- **엔드포인트**: 블록체인 및 TEE 엔드포인트
- **인증**: 키 관리 및 인증 설정
- **동기화**: 동기화 파라미터 조정
- **로깅**: 로그 레벨 및 출력 설정

이 문서는 Pherry 릴레이어 시스템의 구현 세부사항을 다룹니다. 다음 문서에서는 SideVM 격리 실행 환경을 분석합니다.